Tutorial 07

▼ Overview

Tutorial (More Graphs)

- Weighted and Directed Graphs
- Graph Traversal
- Graph Problems

Lab (From Page)

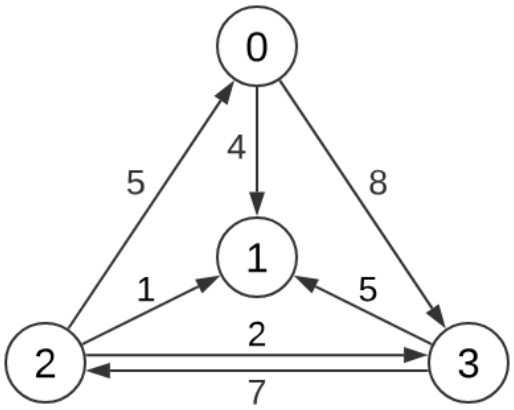
- To explore an application of graphs (Tutorial)
- To get some practice implementing graph search algorithms (Tutorial)
- To perform complexity analysis on graph algorithms (Tutorials Since Week 2)
- To understand the difference between BFS and DFS (Tutorial)

▼ Major Announcements

- Assignment 1 was due Monday
  - Congratulations to anyone who completed
  - Keep going and submit if you are not yet completed as the penalty will raise each day
- AT2 Released Wednesday Week 7 → Today
  - Due 8pm Friday Week 10

▼ Weighted and Directed Graphs

Give the adjacency matrix and adjacency list representation of the following graph:



Adjacency Matrix

▼ What is an adjacency matrix? (Review from last week)

A matrix (table) used to represent the connections in a graph

▼ How big is the matrix?

4 ∗ 4 starting at 0

▼ Fill in the Matrix (Pass)

	0	1
0		
1		
2		
3		

▼ Answer

	0	1
0	0	4
1	0	0
2	5	1

	0	1
3	0	5

▼ What is an adjacency list? (Review from last week)

A list representation of a graph where there exists a list of connections from a node to a node.

▼ What extra information should be stored in each node of the adjacency list now?

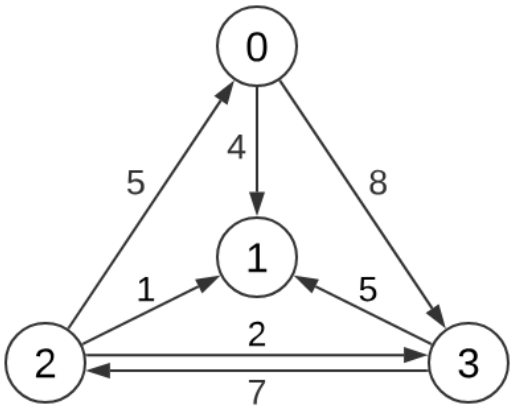
The weight (and the direction which is encoded in the graph.

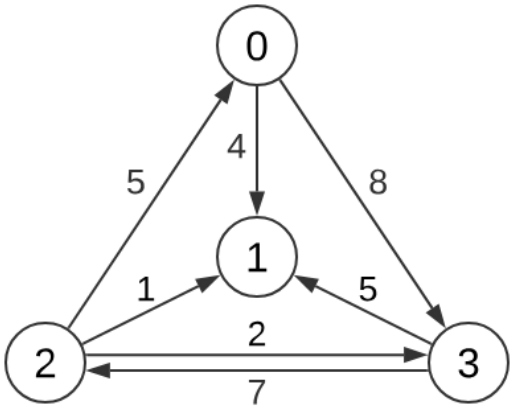
▼ Fill in this adjacency list

			Noc
0		→	
1		→	
2		→	
3		→	

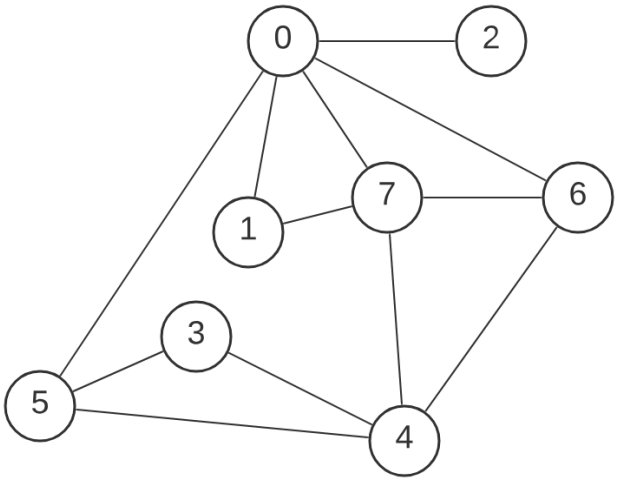
▼ Answer

0		→
1	NULL	→
2		→
3		→





▼ Graph Traversal




```
bfs(G,src):
    Input: graph G, vertex src

  create visited array, initialise
d to false
```

```
dfs(G, src):
    Input: graph G, vertex src

  create visited array, initialised to false
  create predecessor array, initialised to -1
```

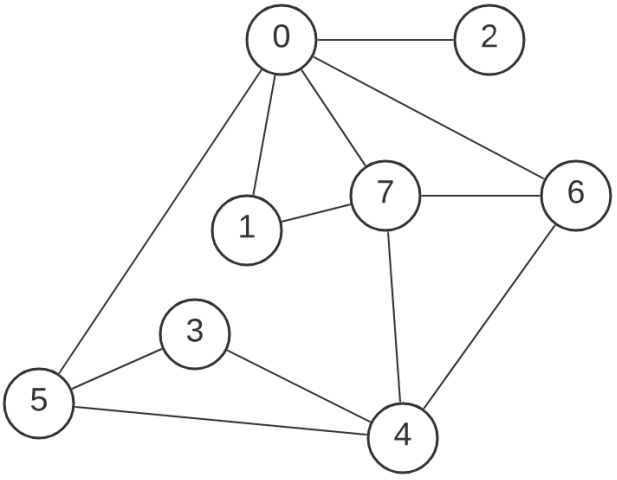
```
  create predecessor array, initia
lised to -1
  create queue Q

  mark src as visited
  enqueue src into Q

    while Q is not empty:
         dequeue v from Q

            for each neighbour w o
f v:
                if w has not been
visited:
                mark w as visited
                set predecessor of
w to v
                enqueue w into Q
```

```
  create stack S

  push src onto S

    while S is not empty:
      pop v from S
      if v has been visited:
                continue

      mark v as visited

            for each neighbour w of v
              (in decreasing order):
                    if w has not been visite
d:
                        set predecessor of w
to v
            push w onto S
```

Trace the execution of the traversal algorithms, and show the state of the `visited` and `pred` arrays and the `Queue` (BFS) or `Stack` (DFS) at the end of each iteration, for each of the following function calls:

▼ bfs(g, 0);

#	Printed	visited		
		0	1	2
0				




```
bfs(G,src):
    Input: graph G, vertex s
rc

  create visited array, init
ialised to false
  create predecessor array,
initialised to -1
  create queue Q

  mark src as visited
  enqueue src into Q

    while Q is not empty:
        dequeue v from Q

            for each neighbo
ur w of v:
                if w has not
been visited:
            mark w as visite
d
            set predecessor
of w to v
            enqueue w into Q
```

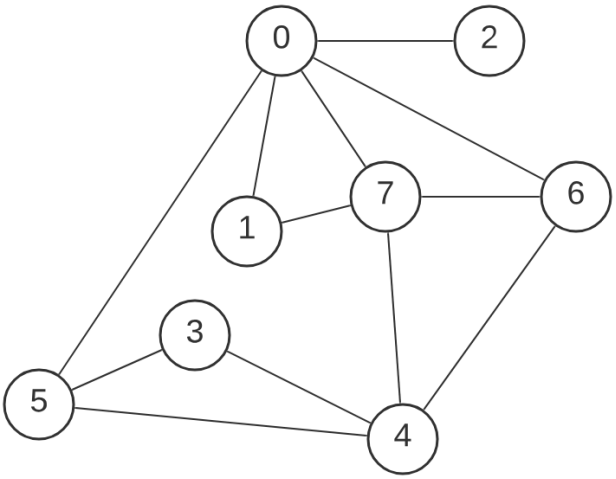
▼ Answer

#	printed	visited				
		0	1	2	3	4
0		1	0	0	0	0
1	0	1	1	1	0	0
2	1	1	1	1	0	0
3	2	1	1	1	0	0
4	5	1	1	1	1	1
5	6	1	1	1	1	1
6	7	1	1	1	1	1
7	3	1	1	1	1	1
8	4	1	1	1	1	1

```
bfs(g, 3);
dfs(g, 0);
dfs(g, 3);
```

▼ bfs(g, 3); (Pass Row)

#	Printed	visited		
		0	1	2
0				




```
bfs(G,src):
    Input: graph G, vertex s
rc

  create visited array, init
ialised to false
  create predecessor array,
initialised to -1
  create queue Q

  mark src as visited
  enqueue src into Q

    while Q is not empty:
        dequeue v from Q

            for each neighbo
ur w of v:
                if w has not
been visited:
            mark w as visite
d
            set predecessor
of w to v
            enqueue w into Q
```

▼ Answer

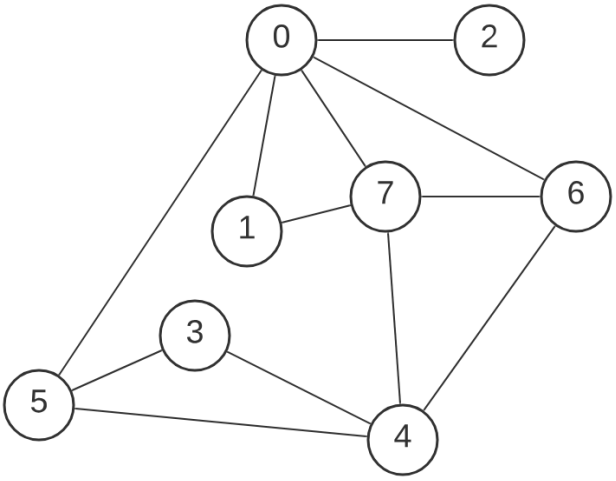
#	printed	visited				
		0	1	2	3	4
0	-	0	0	0	1	0
1	3	0	0	0	1	1
2	4	0	0	0	1	1

3	5	1	0	0	1	1
4	6	1	0	0	1	1
5	7	1	1	0	1	1
6	0	1	1	1	1	1
7	1	1	1	1	1	1
8	2	1	1	1	1	1

```
bfs(g, 3);
dfs(g, 0);
dfs(g, 3);
```

▼ dfs(g, 0);

#	Printed	visited		
		0	1	2
0				




```
dfs(G, src):
    Input: graph G, vertex s
rc

  create visited array, init
ialised to false
  create predecessor array,
initialised to -1
  create stack S

  push src onto S

    while S is not empty:
      pop v from S
      if v has been visited:
                continue

      mark v as visited

            for each neighbo
ur w of v
              (in decreasing
order):
                    if w has
not been visited:
                        set
predecessor of w to v
            push w onto S
```

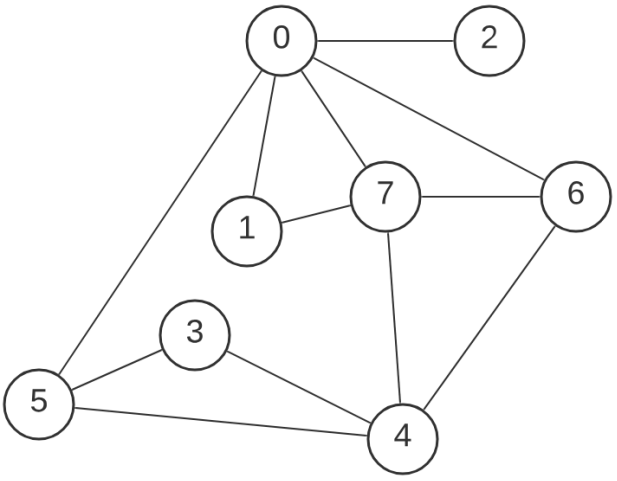
▼ Answer (Pass Rows)

#	printed	visited				
		0	1	2	3	4
0		0	0	0	0	0
1	0	1	0	0	0	0
2	1	1	1	0	0	0
3	7	1	1	0	0	0
4	4	1	1	0	0	1
5	3	1	1	0	1	1
6	5	1	1	0	1	1

7	6	1	1	0	1	1
8	2	1	1	1	1	1

▼ dfs(g, 3); (Pass Row)

#	Printed	visited		
		0	1	2
0				




```
dfs(G, src):
    Input: graph G, vertex s
rc

  create visited array, init
ialised to false
  create predecessor array,
initialised to -1
  create stack S

  push src onto S

    while S is not empty:
      pop v from S
      if v has been visited:
                continue

      mark v as visited

            for each neighbo
ur w of v
              (in decreasing
order):
                    if w has
not been visited:
                        set
predecessor of w to v
            push w onto S
```

▼ Answer (Pass Rows)

#	printed	visited				
		0	1	2	3	4
0	-	0	0	0	0	0
1	3	0	0	0	1	0
2	4	0	0	0	1	1
3	5	0	0	0	1	1
4	0	1	0	0	1	1
5	1	1	1	0	1	1
6	7	1	1	0	1	1
7	6	1	1	0	1	1
8	2	1	1	1	1	1

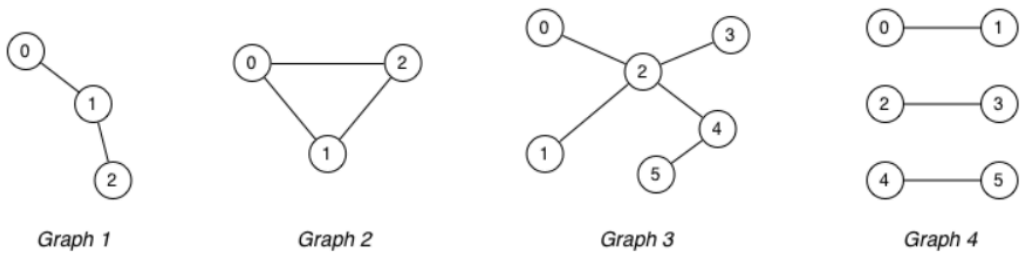
▼ Graph Problems

▼ What is the difference between a *connected* graph and a *complete* graph? Give simple examples of each.

- A connected graph is where all vertices are reachable from the others
  - Have a single connected component.

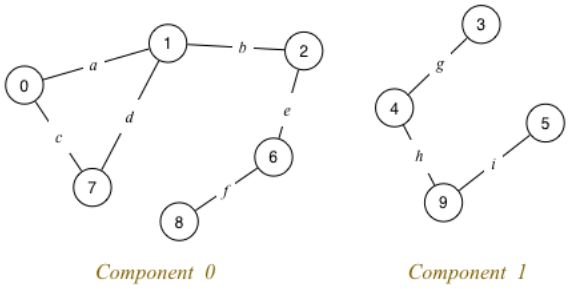
- A complete graph is where all vertices are **directly** connected by a single edge to all others.
  - Have V - 1 incident edges at each vertex where V is the number of vertices (clearly as each edge must connect to all other vertices but not itself).

To be complete a graph must be connected but a connected graph does not have to be complete.



- 1, 2, 3 are connected
- 2 is complete
- 4 is neither

▼ Connected Components Arrays

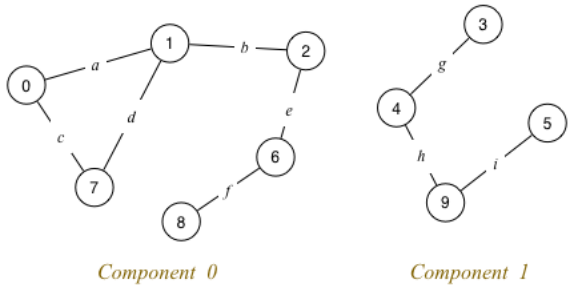


▼ What is a connected components array?

An array that contains integers for each node where the integer corresponds with the component and the index the node.

Consider the connected components array.

`cc[] = {0, 0, 0, 1, 1, 1, 0, 0, 0, 1}`



▼ How would cc change if d were removed?

New cc[] = { }

▼ Answer

cc[] = {0, 0, 0, 1, 1, 1, 0, 0, 0, 1} (Unchanged)

▼ How would cc change if b were removed

New cc[] = { }

▼ Answer

cc[] = {0, 0, 2, 1, 1, 1, 2, 0, 2, 1}

▼ Solving Abstract Problems

Suppose you have n variables $(x_1, x_2, ..., x_n)$, m equalities of the form $x_i = x_j$ (where $i \neq j$) and k inequalities of the form $x_i \neq x_j$ (where $i \neq j$).
Come up with an algorithm (described in plain English or pseudocode) to determine whether this system of equations is consistent or contradictory.

Example

1. Suppose you have three variables $(x_1, x_2, ..., x_n)$, the equality $x_1 = x_2$ and 2 inequalities $x_1 \neq x_3$ and $x_2 \neq x_3$.
   - Consistent as

   $$x_1 = x_2 = 2$$

   and

   $$x_3 = 2$$

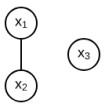
   Or if we set any value of x_i for $i \in \{1, 2, 3\}$ then the other values follow and do not contradict the initial assignment / other assignments.
2. Suppose you have three variables $(x_1, x_2, ..., x_n)$, the equalities $x_1 = x_2$ and $x_1 = x_3$ and 1 inequality $x_2 \neq x_3$.
   - Inconsistent as when you set

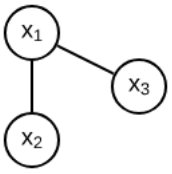
   $$\begin{gathered} x_1 = 1 \\ \implies x_2 = x_1 = 1, \quad x_3 = x_1 = 1 \\ \text{But} \quad x_2 \neq x_3 \end{gathered}$$

   From the third inequality so clearly this cannot be true.

▼ Hint (Reasoning for the algorithm) (The algorithm)

Use a graph, vertices are the x_i and edges are those that are equal, e.g.





Notice that to satisfy correctly all nodes in a connected component must connect to all other nodes in that component otherwise not satisfied.

▼ Answer in Algorithm Form

- Construct a graph from the variables and equalities where:
  - Each vertex represents a variable
  - For each equality xi=xj, insert an edge between i and j
- Compute the connected components array

- For each inequality xi≠xj:
  - If xi and xj are in the same connected component, then return false
- Return true

▼ What is the difference between a Hamiltonian path/circuit and an Euler path/circuit?

▼ Hamiltonian Path

Visits each vertex exactly once.

▼ Hamiltonian Circuit

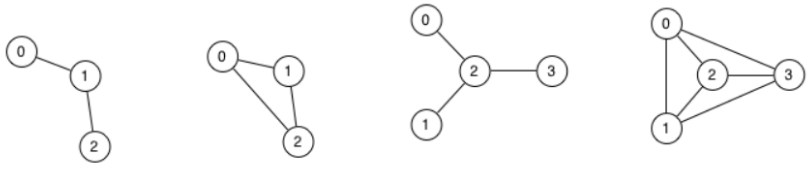
A Hamiltonian circuit is a Hamiltonian path that starts and ends at the same vertex.

▼ Euler Path

Visits each edge exactly once.

▼ Euler Circuit

A Euler circuit is an Euler path that starts and ends at the same vertex.



▼ Identify any Hamiltonian/Euler paths/circuits in the following graphs:

Graph 1 , both paths, circuits as there are no cycles.
Graph 2, both paths, both circuits as there no cycles.

Graph 3, neither paths nor circuits

Graph 4, Hamiltonian path and Circuit no Euler path or circuit mainly because 4 edges with odd degree

▼ Write a function to check whether a path, supplied as an array of `struct edge`s is an Euler path. Assume the function has interface:

`bool isEulerPath(Graph g, struct edge e[], int nE)`

where `e[]` is an array of `nE` edges, in path order.

```
bool isEulerPath(Graph g, struct edge e[], int nE) {

}
```

▼ Hints

- Check that the graph contains all edges because the number of edges given might be incorrect
- Check all the edges are actually in the graph e.g. g→edges[e[i].v because the edges might not be in the graph
- Check whether the path actually is a path, e.g does the ending edge of the last edge have the same vertex as the starting of the next
- Finally are all edges done exactly once
  - Need to check backwards and forwards basically a double loop through the edge array.

▼ Answer

```
// check whether a given path is a Euler path
bool isEulerPath(Graph g, struct edge e[], int nE) {
    assert(g != NULL);

    // includes all edges
    if (g->nE != nE) {
        return false;
    }

    // edges are actually in the graph
    for (int i = 0; i < nE; i++) {
        if (g->edges[e[i].v][e[i].w] == 0) {
            return false;
        }
    }

    // is actually a path
    for (int i = 0; i < nE - 1; i++) {
        if (e[i].w != e[i + 1].v) {
            return false;
        }
    }

    // includes edges exactly once
    for (int i = 0; i < nE; i++) {
        for (int j = i + 1; j < nE; j++) {
            if (e[i].v == e[j].v && e[i].w == e[j].w) return false;
            if (e[i].v == e[j].w && e[i].w == e[j].v) return false;
        }
    }

    return true;
}
```